OBJECT-ORIENTED PROGRAMMING IN JAVA

Formal Lab Report

Course: Object-Oriented Programming
Student Name: Ryan Matsenhura
Reg Number: H250392P

Contents

- Lab 1: Classes and Objects
- Lab 2: Constructors, this, and static Members
- Lab 3: Encapsulation and Access Modifiers
- Lab 4: Inheritance
- Lab 5: Polymorphism
- Lab 6: Abstraction, Abstract Classes and Interfaces
- Lab 7: Exception Handling
- Lab 8: Generics and the Collections Framework
- Lab 9: Nested Types, Enums, Records
- Lab 10: Lambdas and Streams
- Lab 11: Design Patterns and Mini-Project

LAB 1: Classes and Objects

1. Objectives

- Define a class with fields (state) and methods (behaviour).
- Create objects using the new keyword and access members with the dot operator.
- Understand that object variables hold references, not the objects themselves.
- Recognise null and avoid NullPointerException.

2. Source Code

Car.java

```java
package Lab1;

public class Car {  Ryan
    String brand;  3 usages
    int year;  4 usages
    double mileage;  3 usages

    void display() { System.out.println(brand + " (" + year + ") - Mileage: " + mileage + "km"); }

    boolean isAntique() {  2 usages  Ryan
        return (2026 - year) > 25;
    }

    public static void main(String[] args) {  Ryan
        Car c1 = new Car();
        c1.brand = "Toyota Corolla"; c1.year = 2015; c1.mileage = 85000;

        Car c2 = new Car();
        c2.brand = "Ford Mustang"; c2.year = 1967; c2.mileage = 120000;

        c1.display(); System.out.println("Is Antique? " + c1.isAntique());
        c2.display(); System.out.println("Is Antique? " + c2.isAntique());
    }
}
```

Rectangle.java

```java
public class Rectangle {  new *
    public static void main(String[] args) {  new *
            if (r.area() > largest.area()) {
                largest = r;
            }
        }
        System.out.println("Largest Area: " + largest.area() + " (W:" + largest.width + ", H:" + largest.height + ")");

        // --- Intermediate 2: Aliasing Prediction ---
        System.out.println("\n--- Aliasing Test ---");
        Rectangle r1 = new Rectangle();
        r1.width = 10;

        Rectangle r2 = r1; // EXPLAIN: Copies the reference, not the object. They are aliases.
        r2.width = 99;

        // Prediction: r1.width will be 99 because r1 and r2 point to the EXACT SAME object in memory.
        System.out.println("r1.width is: " + r1.width);
    }
}
```

Person.java

```java
class Address {  2 usages  new *
    String city;  3 usages
}
public class Person {  new *
    String name;  5 usages
    Address address;  4 usages
    public static void main(String[] args) {  new *
        // 1. Working chained expression
        Person p1 = new Person();
        p1.name = "Nyasha";
        p1.address = new Address();
        p1.address.city = "Harare";

        // EXPLAIN: Evaluates left to right: p1 -> gets address object -> gets city string
        System.out.println(p1.name + " lives in " + p1.address.city);

        // 2. Demonstrating NullPointerException
        Person p2 = new Person();
        p2.name = "Tafadzwa";
        // We purposefully FORGET to initialize the address: p2.address = new Address();
        try {
            System.out.println(p2.name + " lives in " + p2.address.city);
        } catch (NullPointerException e) {
            System.out.println("Error: " + p2.name + "'s address is null! Cannot access .city");
        }
```

3. Outputs

Car Output:

```
Toyota Corolla (2015) - Mileage: 85000.0km
Is Antique? false
Ford Mustang (1967) - Mileage: 120000.0km
Is Antique? true
```

Rectangle Output:

```
Largest Area: 56.0 (W:7.0, H:8.0)

--- Aliasing Test ---
r1.width is: 99.0
```

Person Output:

```
Nyasha lives in Harare
Error: Tafadzwa's address is null! Cannot access .city
```

4. Conclusion

This lab demonstrated how classes act as blueprints for creating objects. The Rectangle exercise proved that Java object variables store references (memory addresses) rather than the actual objects. When r2 was assigned to r1, modifying r2 modified r1 because both references pointed to the same object in the heap (aliasing). Furthermore, we observed that attempting to call a method or field on an uninitialized object reference results in a NullPointerException.

LAB 2: Constructors, this, and static Members

1. Objectives

- Initialise objects safely using constructors and chain them using this(...).
- Use this to distinguish instance fields from method parameters.
- Explain the difference between instance members and static members.
- Observe the execution order of static and instance initialiser blocks.

2. Source Code

Circle.java

```java
package Lab2;

public class Circle {  Ryan
    private double radius;  1 usage
    // EXPLAIN: Shared across all Circle objects
    private static int circleCount = 0;  2 usages

    // Constructor with parameter
    public Circle(double radius) {  3 usages  Ryan
        this.radius = radius;
        circleCount++;
    }

    // No-argument constructor
    public Circle() {  1 usage  Ryan
        this( radius: 1.0); // EXPLAIN: Constructor chaining. Calls Circle(double radius)
    }

    public static int getCircleCount() {  1 usage  Ryan
        return circleCount;
    }

    public static void main(String[] args) {  Ryan
        Circle c1 = new Circle( radius: 5.5);
        Circle c2 = new Circle(); // Uses default 1.0
        Circle c3 = new Circle( radius: 10.2);

        System.out.println("Total circles created: " + Circle.getCircleCount());
    }
}
```

BlockPrediction.java

```java
package Lab2;

public class BlockPrediction {  Ryan
    static { System.out.println("Static Block 1"); }
    static { System.out.println("Static Block 2"); }

    { System.out.println("Instance Block 1"); }
    { System.out.println("Instance Block 2"); }

    public BlockPrediction() { System.out.println("Constructor"); }  2 usages

    public static void main(String[] args) {  Ryan
        System.out.println("--- Creating first object ---");
        new BlockPrediction();
        System.out.println("--- Creating second object ---");
        new BlockPrediction();
    }
}
```

Employee java:

```
package Lab2;

public class Employee {
    private static int idGenerator = 0;

    private final int employeeId;
    private String name;
    private String role;

    // EXPLAIN: Instance block runs before ANY constructor
    {
        idGenerator++;
        employeeId = idGenerator;
    }

    // 1. Full constructor
    public Employee(String name, String role) {
        this.name = name;
        this.role = role;
    }

    // 2. Partial constructor
    public Employee(String name) {
        this(name, "Unassigned"); // Chains to full constructor
    }

    // 3. No-arg constructor
    public Employee() {
        this("Unknown"); // Chains to partial constructor
    }

    // 4. Copy constructor
    public Employee(Employee other) {
        this.name = other.name;
        this.role = other.role;
        // EXPLAIN: It still gets a UNIQUE ID because the instance block
increments idGenerator
    }

    public void display() {
        System.out.println("ID: " + employeeId + " | Name: " + name + " |
Role: " + role);
    }

    public static void main(String[] args) {
        Employee e1 = new Employee("Alice", "Manager");
        Employee e2 = new Employee("Bob");
        Employee e3 = new Employee(e1); // Copy of Alice

        e1.display();
        e2.display();
        e3.display(); // Will have same name/role as e1, but ID will be 3
    }
}
```

Temperature.java:

```java
package Lab2;

public class Temperature {  Ryan
    private final double kelvin; // Storing internally as a single source of truth  2 usa

    // EXPLAIN: Private constructor forces callers to use the static methods
    private Temperature(double kelvin) {  2 usages  Ryan
        this.kelvin = kelvin;
    }

    // EXPLAIN: Static factory methods
    public static Temperature fromCelsius(double celsius) {  1 usage  Ryan
        return new Temperature( kelvin: celsius + 273.15);
    }

    public static Temperature fromFahrenheit(double fahrenheit) {  1 usage  Ryan
        return new Temperature( kelvin: (fahrenheit - 32) * 5.0 / 9.0 + 273.15);
    }

    public double getCelsius() { return kelvin - 273.15; }  2 usages  Ryan

    public static void main(String[] args) {  Ryan
        Temperature t1 = Temperature.fromCelsius(25);
        Temperature t2 = Temperature.fromFahrenheit(77);

        System.out.printf("T1 in Celsius: %.2f%n", t1.getCelsius());
        System.out.printf("T2 in Celsius: %.2f%n", t2.getCelsius());
    }
}
```

3. Outputs

Circle Output:

```
Total circles created: 3
```

BlockPrediction Output:

```
Static Block 1
Static Block 2
--- Creating first object ---
Instance Block 1
Instance Block 2
Constructor
--- Creating second object ---
Instance Block 1
Instance Block 2
Constructor
```

Employee output:

```
ID: 1 | Name: Alice | Role: Manager
ID: 2 | Name: Bob | Role: Unassigned
ID: 3 | Name: Alice | Role: Manager
```

Temperature output:

```
T1 in Celsius: 25.00
T2 in Celsius: 25.00
```

4. Conclusion

This lab highlighted the object lifecycle. We utilized constructors for safe initialisation and reduced code duplication through constructor chaining with this(). We also verified that static fields are shared across all class instances. Finally, the block prediction exercise proved the execution flow: static blocks run exactly once when the class is loaded by the JVM, while instance blocks run every time the new keyword is used, immediately preceding the constructor body.

LAB 3: Encapsulation and Access Modifiers

1. Objectives

- Hide internal data using private fields and expose it via controlled methods.
- Apply access modifiers correctly to protect object integrity.
- Enforce validation rules inside setters and constructors to prevent invalid states.
- Understand the dangers of returning mutable objects (defensive copying).

2. Source Code

Student.java

```
package lab3;

public class Student {
    private String name;
    private int marks;

    public Student(String name, int marks) {
        this.name = name;
        setMarks(marks); // Enforce validation at creation
    }

    public void setMarks(int marks) {
        if (marks < 0 || marks > 100) {
            throw new IllegalArgumentException("Marks must be between 0 and 100");
        }
        this.marks = marks;
    }
    public String getGrade() {
        return marks >= 80 ? "A" : "F";
    }

    public static void main(String[] args) {
        Student s = new Student("Tatenda", 85);
        System.out.println("Grade: " + s.getGrade());
    }
}
```

BankAccount.java *(Defensive Copying Snippet)*

```
package lab3;
import java.util.*;

public class BankAccount {
    private final List<String> history = new ArrayList<>();

    // Returns an unmodifiable list so external classes cannot bypass encapsulation
    public List<String> getHistory() {
        return Collections.unmodifiableList(history);
    }
}
```

3. Outputs

Student Output:

```
Grade: A
```

(If initialized with marks = 105, the program correctly throws IllegalArgumentException: Marks must be between 0 and 100).

4. Conclusion

Encapsulation effectively transformed our classes into "black boxes." By making fields private and routing data changes through setter methods, we guaranteed that objects (like Student marks) cannot enter an invalid state. Additionally, we learned that simply making a field private is not enough if a getter returns a reference to a mutable object (like an ArrayList). Using Collections.unmodifiableList() prevents external interference, maintaining true data security.

LAB 4: Inheritance

1. Objectives

- Create subclasses using extends to reuse superclass code.
- Call superclass constructors and methods utilizing super.
- Understand constructor call order across multiple inheritance levels.
- Differentiate between inheritance ("is-a" relationship) and composition ("has-a" relationship).

2. Source Code

SchoolDemo.java

```
package lab4;

class Person {
    private String name;
    public Person(String name) { this.name = name; }
    public void display() { System.out.print("Name: " + name); }
}

class Student extends Person {
    private String course;
    public Student(String name, String course) {
        super(name); // Must be the first line
        this.course = course;
    }
    @Override
    public void display() {
        super.display(); // Reuse parent logic
        System.out.println(" | Course: " + course);
    }
}

public class SchoolDemo {
    public static void main(String[] args) {
        Student s = new Student("Chipo", "Computer Science");
        s.display();
    }
}
```

StackDemo.java *(Composition over Inheritance)*

```
package lab4;
import java.util.ArrayList;

class GoodStack<T> {
    private final ArrayList<T> list = new ArrayList<>(); // Composition (HAS-A)
    public void push(T item) { list.add(item); }
    public T pop() { return list.remove(list.size() - 1); }
}
```

3. Outputs

SchoolDemo Output:

```
Name: Chipo | Course: Computer Science
```

4. Conclusion

Inheritance allowed us to drastically reduce code duplication by establishing hierarchical relationships. The Student class successfully acquired the fields and methods of Person. We observed that a subclass must explicitly call the parent constructor using super(...) if the parent lacks a default no-argument constructor. Most importantly, the Challenge exercise taught us to favour Composition over Inheritance when the relationship is not strictly "is-a". Inheriting ArrayList to make a Stack incorrectly exposed methods like .clear(), while Composition securely encapsulated the list.

LAB 5: Polymorphism

1. Objectives

- Distinguish compile-time polymorphism (overloading) from run-time polymorphism (overriding).
- Use a superclass reference to hold subclass objects (upcasting) and explain dynamic method dispatch.
- Prove that methods follow the actual object type, while fields follow the declared reference type.

2. Source Code

AnimalDemo.java

```
package lab5;

class Animal {
    String type = "Generic Animal"; // Field
    void sound() { System.out.println("..."); }
}
class Cat extends Animal {
    String type = "Cat"; // Hides parent field
    @Override void sound() { System.out.println("Meow"); }
}

public class AnimalDemo {
    public static void main(String[] args) {
        Animal myAnimal = new Cat(); // Upcasting

        // Run-time Polymorphism (Dynamic Dispatch)
        myAnimal.sound();

        // Fields are NOT polymorphic
        System.out.println("Type field: " + myAnimal.type);
    }
}
```

3. Outputs

```
Meow
Type field: Generic Animal
```

4. Conclusion

This lab demonstrated the power of polymorphism. By programming to a superclass reference (Animal myAnimal = new Cat()), our code became highly flexible. The JVM used dynamic method dispatch at run time to ensure the overridden sound() method from the Cat class was executed. However, we discovered a crucial rule: fields and static methods are not polymorphic; they are resolved at compile time based on the declared reference type (Animal), which is why myAnimal.type printed "Generic Animal" instead of "Cat".

LAB 6: Abstraction, Abstract Classes and Interfaces

1. Objectives

- Define abstract classes and abstract methods, understanding why they cannot be instantiated.
- Define and implement interfaces, managing multiple inheritance of type.
- Program to an interface, promoting loose coupling.

2. Source Code

ShapeDemo.java

```
package lab6;

abstract class Shape {
    abstract double area(); // No body, forces subclasses to implement
}
class Circle extends Shape {
    double radius;
    Circle(double radius) { this.radius = radius; }
    @Override double area() { return Math.PI * radius * radius; }
}
class Rectangle extends Shape {
    double w, h;
    Rectangle(double w, double h) { this.w = w; this.h = h; }
    @Override double area() { return w * h; }
}

public class ShapeDemo {
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(5), new Rectangle(4, 6) };
        for (Shape s : shapes) {
            System.out.printf("%s Area: %.2f%n", s.getClass().getSimpleName(),
s.area());
        }
    }
}
```

3. Outputs

```
Circle Area: 78.54
Rectangle Area: 24.00
```

4. Conclusion

Abstraction allowed us to hide implementation details and focus on what an object does rather than how it does it. Abstract classes provided a shared template for related subclasses (Circle and Rectangle), ensuring a strict contract (the area() method) while allowing shared state. We also observed that utilizing interfaces acts as a pure contract of capabilities, overcoming Java's restriction against multiple inheritance for classes.

LAB 7: Exception Handling

1. Objectives

- Differentiate between checked exceptions (compile-time) and unchecked exceptions (runtime).
- Use try, catch, finally, throw, and throws correctly.
- Create custom exceptions to carry domain-specific error information.

2. Source Code

BankDemo.java

```
package lab7;

class InsufficientFundsException extends Exception { // Custom Checked Exception
    public InsufficientFundsException(String msg) { super(msg); }
}

class BankAccount {
    private double balance = 100.0;

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException("Shortfall of $" + (amount -
balance));
        }
        balance -= amount;
        System.out.println("Withdrew $" + amount);
    }
}

public class BankDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        try {
            acc.withdraw(150);
        } catch (InsufficientFundsException e) {
            System.out.println("Transaction failed: " + e.getMessage());
        } finally {
            System.out.println("Transaction attempt logged.");
        }
    }
}
```

3. Outputs

```
Transaction failed: Shortfall of $50.0
Transaction attempt logged.
```

4. Conclusion

Exception handling decoupled our core logic from error-handling routines. By extending Exception, we created a custom checked exception (InsufficientFundsException), forcing any calling method to handle the potential failure via a try-catch block. We also verified that the finally block consistently executes to clean up resources, regardless of whether an exception was thrown or caught.

LAB 8: Generics and the Collections Framework

1. Objectives

- Write generic classes to achieve compile-time type safety and avoid explicit casts.
- Choose appropriate collections: List (ordered), Set (unique), and Map (key-value pairs).

2. Source Code

GenericStack.java

```
package lab8;
import java.util.ArrayList;

public class GenericStack<T> {
    private final ArrayList<T> elements = new ArrayList<>();
    public void push(T item) { elements.add(item); }
    public T pop() { return elements.remove(elements.size() - 1); }

    public static void main(String[] args) {
        GenericStack<String> stack = new GenericStack<>();
        stack.push("Java");
        stack.push("OOP");
        System.out.println("Popped: " + stack.pop());
    }
}
```

3. Outputs

```
Popped: OOP
```

4. Conclusion

Generics resolved the issue of ClassCastExceptions at runtime by enforcing type constraints at compile time (Type Erasure). By designing a GenericStack<T>, the class became reusable for any reference type (Strings, Integers, or custom objects). Utilizing the Java Collections Framework (specifically List, Set, and Map) allowed for efficient data storage, manipulation, and retrieval without reinventing data structures from scratch.

LAB 9: Nested Types, Enums, Records

1. Objectives

- Model fixed sets of constants with enum.
- Create immutable data carriers seamlessly using record.
- Ensure the equals and hashCode contract is maintained.

2. Source Code

EnumRecordDemo.java

```
package lab9;

enum Level { LOW, MEDIUM, HIGH }

// Record automatically generates constructor, getters, equals, hashCode, and toString
record Student(String name, int age) {}

public class EnumRecordDemo {
    public static void main(String[] args) {
        Level current = Level.HIGH;
        System.out.println("Current Level: " + current);

        Student s1 = new Student("Tariro", 21);
        Student s2 = new Student("Tariro", 21);

        System.out.println(s1);
        System.out.println("Are students equal by value? " + s1.equals(s2));
    }
}
```

3. Outputs

```
Current Level: HIGH
Student[name=Tariro, age=21]
Are students equal by value? true
```

4. Conclusion

Enums provided type safety for variables that should only accept a predefined set of values, eliminating the risks of using arbitrary strings or integers. Records (introduced in modern Java) proved highly efficient for designing plain data carriers, automatically providing correct implementations for equals, hashCode, and toString. This ensures that two separate record objects with identical data evaluate as mathematically equal.

LAB 10: Lambdas and Streams

1. Objectives

- Write lambda expressions as implementations for functional interfaces.
- Process collections declaratively utilizing the Stream API (filter, map, sorted, collect).

2. Source Code

StreamDemo.java

```
package lab10;
import java.util.*;

record Student(String name, double mark) {}

public class StreamDemo {
    public static void main(String[] args) {
        List<Student> students = List.of(
            new Student("Alice", 60), new Student("Bob", 85),
            new Student("Charlie", 40), new Student("Diana", 90)
        );

        double avg = students.stream().mapToDouble(Student::mark).average().orElse(0);
        System.out.println("Class Average: " + avg);

        System.out.println("Students above average:");
        students.stream()
            .filter(s -> s.mark() > avg)
            .sorted(Comparator.comparing(Student::name))
            .forEach(s -> System.out.println("- " + s.name()));
    }
}
```

3. Outputs

```
Class Average: 68.75
Students above average:
- Bob
- Diana
```

4. Conclusion

Lambdas and the Stream API revolutionized the way we handle data manipulation. Instead of writing verbose for loops and if statements (imperative programming), streams allowed for declarative processing pipelines. Operations like filter and map successfully demonstrated how complex collection querying can be written safely and concisely.

LAB 11: Design Patterns and Mini-Project

1. Objectives

- Apply SOLID principles and recognise standard Design Patterns (Singleton, Factory, Observer, Strategy).
- Combine all core OOP concepts (Encapsulation, Inheritance, Polymorphism, Abstraction, Collections, Exceptions, and Streams) into a comprehensive Library Management System.

2. Source Code

LibrarySystem.java *(Core Service Snippet)*

```
package lab11;
import java.util.*;

// Interface and Abstract implementation showing Abstraction & Polymorphism
interface Borrowable { void borrow(Member m) throws Exception; }

abstract class LibraryItem implements Borrowable {
    private String id, title;
    private boolean available = true;

    protected LibraryItem(String id, String title) { this.id = id; this.title = title;
}
    public String getTitle() { return title; }

    @Override
    public void borrow(Member m) throws Exception {
        if (!available) throw new Exception(title + " is unavailable.");
        available = false;
    }
}

class Book extends LibraryItem {
    public Book(String id, String title) { super(id, title); }
}
class Member {
    private String name;
    public Member(String name) { this.name = name; }
    public String getName() { return name; }
}

public class LibrarySystem {
    public static void main(String[] args) {
        try {
            LibraryItem book = new Book("B1", "Effective Java");
            Member m1 = new Member("Kenneth");

            book.borrow(m1); // Successful borrow
            System.out.println(m1.getName() + " borrowed " + book.getTitle());

            book.borrow(m1); // Exception triggers here
        } catch (Exception e) {
            System.out.println("ERROR: " + e.getMessage());
        }
    }
}
```

3. Outputs

```
Kenneth borrowed Effective Java
ERROR: Effective Java is unavailable.
```

4. Conclusion

The mini-project successfully validated the core principles of OOP learned throughout the manual. Abstraction and Interfaces established a robust framework, while Encapsulation protected internal statuses like item availability. Polymorphism enabled treating Books and DVDs uniformly as LibraryItems. By integrating Custom Exceptions and Data Collections, the software became robust and scalable, proving that applying sound software engineering practices yields maintainable, real-world solutions.